

Manual Técnico





Índice

01 Desarrollo

- 01. Visión General de la Arquitectura
- 01.2 Diagrama de la Arquitectura
- 01.3 Infraestructura y Despliegue
- 02.1 Organización de Carpetas
- 02.2 Componentes Clave y Principios Arquitectónicos Controladores
- 0.3 Estructura del Código Frontend
- 0.3.1 Organización de Carpetas
- 0.3.2 Componentes Clave Servicios Core
- 0.4. Funcionalidades Destacadas
- 0.4.1 Comunicación en Tiempo Real
- 0.4.2 Autenticación y Autorización
- 0.4.3 Manejo de Errores

02 Despliegue

- 01. Despliegue de la Solución en Azure - Sistema de Gestión de Máquinas Virtuales
- 1.1 Arquitectura de Despliegue
- 1.2 Componentes de la Infraestructura del Proyecto VM
- 1.2.1 Frontend
- 1.2.2 Backend
- 1.2.3 Base de Datos
- 1.2.4 Máquina Virtual de Gestión
- 1.2.5 Redes y Seguridad para el Sistema VM



01

Desarrollo

01. Arquitectura del sistema

El sistema de Gestión de Máquinas Virtuales implementa una arquitectura distribuida cliente-servidor moderna, estableciendo capas claramente diferenciadas que favorecen el mantenimiento, la escalabilidad y la seguridad.

1.1 Visión General de la Arquitectura

La aplicación se compone de dos elementos principales que se comunican mediante protocolos estándar:

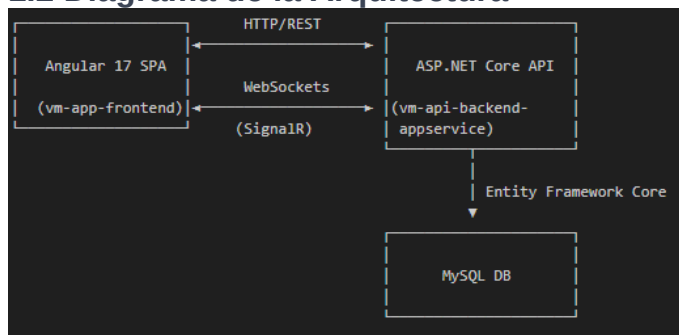
Frontend (Cliente)

- **Tecnología:** Angular 17 con TypeScript
- **Patrón arquitectónico:** Arquitectura basada en componentes con servicios inyectables
- **Protocolos de comunicación:**
- HTTP/REST para operaciones CRUD
- WebSockets para actualizaciones en tiempo real

Backend (Servidor)

- **Tecnología:** ASP.NET Core 7+
- **Patrón arquitectónico:** Arquitectura en capas con separación de responsabilidades
- **Persistencia:** Entity Framework Core con MySQL
- **Comunicación en tiempo real:** SignalR

1.2 Diagrama de la Arquitectura



1.3 Infraestructura y Despliegue

El sistema está diseñado para ser desplegado en la nube de Microsoft Azure:

- **Backend:** Azure App Service
- **Frontend:** Azure Static Web Apps
- **Base de datos:** Azure Database for MySQL

2.1 Organización de Carpetas

```
vm-api-backend-appservice/  
├── Controllers/           # API endpoints y gestión de solicitudes HTTP  
├── Services/             # Lógica de negocio y orquestación  
│   ├── IVirtualMachineService.cs  
│   ├── VirtualMachineService.cs  
│   ├── IVirtualMachineNotificationService.cs  
│   └── VirtualMachineNotificationService.cs  
├── Repositories/         # Acceso a datos  
│   ├── IVirtualMachineRepository.cs  
│   └── VirtualMachineRepository.cs  
├── Models/               # Entidades y DTOs  
│   ├── VirtualMachine.cs # Entidad principal  
│   └── DTOs/              # Objetos de transferencia de datos  
├── Data/                 # Contexto EF Core y configuración de DB  
│   └── VmDbContext.cs  
├── Hubs/                  # Endpoints SignalR para tiempo real  
│   └── VirtualMachineHub.cs  
├── Middleware/            # Middleware personalizado  
│   ├── ExceptionHandlingMiddleware.cs  
│   └── JwtAuthMiddleware.cs  
├── Exceptions/           # Excepciones personalizadas  
│   ├── NotFoundException.cs  
│   └── BadRequestException.cs  
└── Program.cs             # Configuración de la aplicación
```

2.2 Componentes Clave y Principios Arquitectónicos

Controladores

Los controladores definen los endpoints REST y gestionan las solicitudes HTTP. Ejemplo de código del controlador principal:

```
// Ruta: Controllers/VirtualMachinesController.cs  
[ApiController]  
[Route("api/vms")]  
[Produces("application/json")]  
public class VirtualMachinesController : ControllerBase  
{  
    private readonly IVirtualMachineService _vmService;  
    private readonly ILogger<VirtualMachinesController> _logger;  
  
    public VirtualMachinesController(  
        IVirtualMachineService vmService,  
        ILogger<VirtualMachinesController> logger)  
    {  
        _vmService = vmService;  
        _logger = logger;  
    }  
  
    [HttpGet]  
    [ProducesResponseType(typeof(IEnumerable<VirtualMachineResponseDto>), 200)]  
    public async Task<ActionResult<IEnumerable<VirtualMachineResponseDto>>> GetAllVirtualMachines()  
    {  
        try  
        {  
            var vms = await _vmService.GetAllVirtualMachinesAsync();  
            return Ok(vms);  
        }  
        catch (Exception ex)  
        {  
            _logger.LogError(ex, "Error getting all virtual machines");  
            return StatusCode(500, new { error = "An error occurred while retrieving virtual machines" });  
        }  
    }  
  
    // Otros endpoints (GET, POST, PUT, DELETE)...  
}
```

Este controlador implementa el principio de responsabilidad única al ocuparse exclusivamente de la gestión de las solicitudes HTTP, delegando la lógica de negocio al servicio correspondiente.

Servicios

Los servicios implementan la lógica de negocio y orquestan las operaciones. Se definen mediante interfaces para facilitar el testing y el desacoplamiento:

```
// Ruta: Services/IVirtualMachineService.cs
public interface IVirtualMachineService
{
    Task<IEnumerable<VirtualMachineResponseDto>> GetAllVirtualMachinesAsync();
    Task<VirtualMachineResponseDto> GetVirtualMachineByIdAsync(int id);
    Task<VirtualMachineResponseDto> CreateVirtualMachineAsync(CreateVirtualMachineDto createVmDto);
    Task<VirtualMachineResponseDto> UpdateVirtualMachineAsync(int id, UpdateVirtualMachineDto updateVmDto);
    Task<bool> DeleteVirtualMachineAsync(int id);
}

// Ruta: Services/VirtualMachineService.cs
public class VirtualMachineService : IVirtualMachineService
{
    private readonly IVirtualMachineRepository _vmRepository;
    private readonly VmDbContext _dbContext;
    private readonly IVirtualMachineNotificationService _notificationService;

    public VirtualMachineService(
        IVirtualMachineRepository vmRepository,
        VmDbContext dbContext,
        IVirtualMachineNotificationService notificationService)
    {
        _vmRepository = vmRepository;
        _dbContext = dbContext;
        _notificationService = notificationService;
    }

    public async Task<VirtualMachineResponseDto> CreateVirtualMachineAsync(CreateVirtualMachineDto createVmDto)
    {
        // Validación y lógica de negocio
        await ValidateOperatingSystemIdAsync(createVmDto.OperatingSystemId);
        await ValidateStatusIdAsync(createVmDto.StatusId);

        var vm = new VirtualMachine {
            Name = createVmDto.Name,
            Cores = createVmDto.Cores,
            // Mapeo de otras propiedades...
        };

        var createdVm = await _vmRepository.CreateVirtualMachineAsync(vm);

        if (createdVm == null)
        {
            throw new BadRequestException("Failed to create virtual machine");
        }

        var responseDto = MapToResponseDto(createdVm);
    }
}
```

Esta estructura de servicios permite segregar las responsabilidades, facilitando el mantenimiento y las pruebas unitarias.

Sistema de Notificaciones en Tiempo Real

La implementación de tiempo real se basa en SignalR, con dos componentes principales:

```
// Ruta: Hubs/VirtualMachineHub.cs
[AllowAnonymous]
public class VirtualMachineHub : Hub
{
    private readonly ILogger<VirtualMachineHub> _logger;

    public VirtualMachineHub(ILogger<VirtualMachineHub> logger)
    {
        _logger = logger;
    }

    public override async Task OnConnectedAsync()
    {
        string userId = Context.User?.Identity?.Name ?? "Anonymous";
        _logger.LogInformation($"SignalR client connected: (Context.ConnectionId, User: {userId})");

        // Añadir cliente al grupo de actualizaciones
        await Groups.AddToGroupAsync(Context.ConnectionId, "VmUpdatesGroup");

        await base.OnConnectedAsync();
    }

    // Otros métodos de gestión de conexiones...
}
```

```
// Ruta: Services/VirtualMachineNotificationService.cs
public class VirtualMachineNotificationService : IVirtualMachineNotificationService
{
    private readonly IHubContext<VirtualMachineHub> _hubContext;
    private readonly ILogger<VirtualMachineNotificationService> _logger;

    public VirtualMachineNotificationService(
        IHubContext<VirtualMachineHub> hubContext,
        ILogger<VirtualMachineNotificationService> logger)
    {
        _hubContext = hubContext;
        _logger = logger;
    }

    public async Task NotifyVmCreatedAsync(VirtualMachineResponseDto vm)
    {
        try
        {
            // Enviar a grupo específico y a todos los clientes para redundancia
            var groupTask = _hubContext.Clients.Group("VmUpdatesGroup").SendAsync("VmCreated", vm);
            var allTask = _hubContext.Clients.All.SendAsync("VmCreated", vm);

            await Task.WhenAll(groupTask, allTask);

            _logger.LogInformation($"VM created notification sent: VM ID {vm.Id}");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"Failed to send VM created notification for VM ID: {vm.Id}");
        }
    }

    // Métodos similares para Update y Delete...
}
```

Configuración de la Aplicación

En Program.cs se configura la aplicación, incluyendo la configuración de SignalR:

```
// Ruta: Program.cs
// Configurar SignalR
builder.Services.AddSignalR(options => {
    options.EnableDetailedErrors = true;
    options.MaximumReceiveMessageSize = 102400; // 100 KB
});

// Más adelante en el código...

// Configurar rutas de SignalR con opciones avanzadas
app.MapHub<VirtualMachineHub>("/hubs/virtualmachines", options => {
    options.CloseOnAuthenticationExpiration = false;
    options.TransportMaxBufferSize = 102400;
    options.ApplicationMaxBufferSize = 102400;
    options.WebSockets.CloseTimeout = TimeSpan.FromSeconds(30);
});
```

3. Estructura del Código Frontend

El frontend implementa una arquitectura basada en componentes con servicios reactivos.

3.1 Organización de Carpetas

```
vm-app-frontend/
├── src/
│   ├── app/
│   │   ├── core/ # Núcleo de la aplicación
│   │   │   ├── services/ # Servicios principales
│   │   │   │   ├── auth.service.ts # Autenticación
│   │   │   │   ├── signalr.service.ts # Comunicación en tiempo real
│   │   │   │   └── virtual-machine.service.ts
│   │   │   ├── interceptors/ # Interceptores HTTP
│   │   │   ├── guards/ # Guardias de ruta
│   │   │   └── interfaces/ # Interfaces y tipos
│   │   ├── models/
│   │   │   └── virtual-machine.model.ts
│   │   ├── modules/ # Módulos funcionales
│   │   │   └── virtual-machines/ # Gestión de máquinas virtuales
│   │   │       ├── vm-list/
│   │   │       ├── vm-details/
│   │   │       └── vm-dialog/
│   │   ├── shared/ # Componentes compartidos
│   │   │   ├── components/
│   │   │   └── signalr-debug/
│   └── environments/ # Configuraciones por entorno
```

3.2 Componentes Clave

Servicios Core

Los servicios centrales mantienen la comunicación con la API y el estado de la aplicación:

```
// Ruta: src/app/core/services/virtual-machine.service.ts
@Injectable({
  providedIn: 'root'
})
export class VirtualMachineService implements OnDestroy {
  private apiUrl = `${environment.apiUrl}/api/vms`;
  private vmListSubject = new BehaviorSubject<VirtualMachine[]>([]);
  public vmList$ = this.vmListSubject.asObservable();

  private subscriptions: Subscription[] = [];
  private initialized = false;

  constructor(
    private http: HttpClient,
    private signalrService: SignalrService
  ) {
    // Inicializar conexión SignalR y suscribirse a eventos
    this.signalrService.startConnection()
      .then(() => {
        this.subscribeToRealTimeEvents();
        this.loadInitialData();
      })
      .catch(() => {
        // Cargar datos iniciales incluso si SignalR falla
        this.loadInitialData();
      });

    // Suscribirse a cambios de conexión
    const connectionSub = this.signalrService.connected$.subscribe(connected => {
      if (connected && this.initialized) {
        this.loadInitialData();
      }
    });
    this.subscriptions.push(connectionSub);
  }

  // Métodos CRUD para interactuar con la API...
}
```

```
public startConnection(): Promise<void> {
  return new Promise<void>((resolve, reject) => {
    if (isPlatformBrowser(this.platformId)) {
      // Obtener el token de autenticación
      const token = this.authService.getToken();

      // Construir la conexión SignalR con autenticación
      this.hubConnection = new signalR.HubConnectionBuilder()
        .withUrl(`${environment.apiUrl}/hubs/virtualmachines`, {
          accessTokenFactory: () => token || ''
        })
        .withAutomaticReconnect()
        .configureLogging(signalR.LogLevel.Error)
        .build();

      // Configurar manejadores de eventos de conexión
      this.configureConnectionHandlers();

      // Iniciar la conexión
      this.hubConnection
        .start()
        .then(() => {
          this.connectionStatus.next(true);
          this.registerSignalEvents();
          this.hubConnection?.invoke('JoinVmUpdates')
            .then(resolve)
            .catch(reject);
        })
        .catch(reject);
    } else {
      resolve(); // En SSR, resolver sin hacer nada
    }
  });
}

private registerSignalEvents(): void {
  if (this.hubConnection) {
    // Registrar manejadores para eventos de VM
    this.hubConnection.on('VmCreated', (vm: VirtualMachine) => {
      this.vmCreatedSubject.next(vm);
    });

    // Manejadores similares para Updated y Deleted...
  }
}
```

Servicio SignalR

El servicio SignalR gestiona la conexión en tiempo real:

```
// Ruta: src/app/core/services/signalr.service.ts
@Injectable({
  providedIn: 'root'
})
export class SignalrService {
  private hubConnection: signalR.HubConnection | null = null;
  private vmCreatedSubject = new BehaviorSubject<VirtualMachine | null>(null);
  private vmUpdatedSubject = new BehaviorSubject<VirtualMachine | null>(null);
  private vmDeletedSubject = new BehaviorSubject<number | null>(null);
  private connectionStatus = new BehaviorSubject<boolean>(false);

  // Observables públicos para que los componentes se suscriban
  public vmCreated$ = this.vmCreatedSubject.asObservable();
  public vmUpdated$ = this.vmUpdatedSubject.asObservable();
  public vmDeleted$ = this.vmDeletedSubject.asObservable();
  public connected$ = this.connectionStatus.asObservable();

  constructor(
    @Inject(PLATFORM_ID) private platformId: Object,
    private authService: AuthService
  ) {}
}
```

Componentes de UI

Los componentes Angular implementan la interfaz de usuario, con VmListComponent como principal componente para la visualización y gestión:

```
// Ruta: src/app/modules/virtual-machines/vm-list/vm-list.component.ts
@Component({
  selector: 'app-vm-list',
  templateUrl: './vm-list.component.html',
  styleUrls: ['./vm-list.component.scss']
})
export class VmListComponent implements OnInit, OnDestroy {
  displayedColumns: string[] = ['name', 'cores', 'ram', 'disk', 'os', 'status', 'actions'];
  dataSource = new MatTableDataSource<VirtualMachine>([]);
  isLoading = false;
  private destroy$ = new Subject<void>();

  @ViewChild(MatPaginator) paginator!: MatPaginator;
  @ViewChild(MatSort) sort!: MatSort;

  constructor(
    private vmService: VirtualMachineService,
    private authService: AuthService,
    private translate: TranslateService,
    private dialog: MatDialog,
    private snackBar: MatSnackBar
  ) {}

  ngOnInit(): void {
    this.loadVirtualMachines();

    // Suscribirse a actualizaciones en tiempo real
    this.vmService.vmList$.subscribe(vms => {
      if (vms && vms.length > 0) {
        this.dataSource.data = vms;

        // Resetear paginación cuando los datos se actualizan
        if (this.dataSource.paginator) {
          this.dataSource.paginator.firstPage();
        }
      }
    });
  }

  // Métodos para cargar, crear, editar y eliminar VMs...
}
```

4. Funcionalidades Destacadas

4.1 Comunicación en Tiempo Real

La comunicación en tiempo real es una característica destacada del sistema:

1. Backend:

- Hub SignalR que gestiona conexiones/desconexiones
- Servicio de notificaciones que envía actualizaciones a clientes
- Soporte para grupos de clientes

2. Frontend:

- Servicio SignalR que mantiene la conexión
- Reactivos que propagan cambios a componentes
- Reconexión automática en caso de fallos de red

4.2 Autenticación y Autorización

Sistema de seguridad basado en tokens JWT:

1. Backend:

- Middleware personalizado para validación de tokens
- Atributos de autorización (AdminOnly)
- Integración de JWT con SignalR

2. Frontend:

- Interceptor HTTP para inyectar tokens
- Guardias de ruta para proteger rutas sensibles
- Servicio de autenticación centralizado

4.3 Manejo de Errores

Sistema robusto de manejo de errores:

1. Backend:

- Excepciones tipadas (BadRequestException, NotFoundException)
- Middleware de manejo de excepciones
- Respuestas HTTP con códigos de estado apropiados

2. Frontend:

- Interceptor HTTP para procesar errores de API
- Mensajes de error contextuales
- Reintentos automáticos cuando es apropiado



02

Despliegue

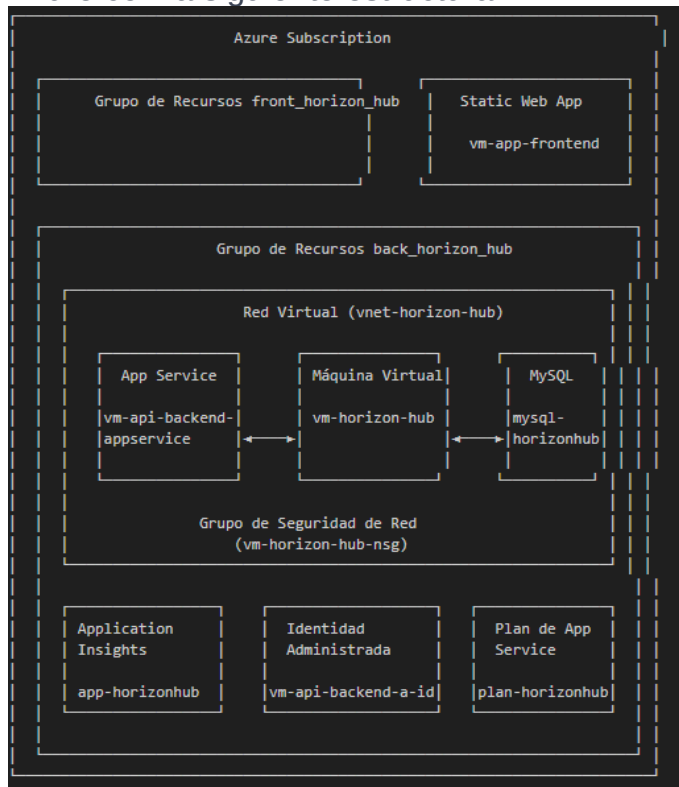


01. Despliegue de la Solución en Azure - Sistema de Gestión de Máquinas Virtuales

La infraestructura de la solución "Sistema de Gestión de Máquinas Virtuales" se ha implementado en Microsoft Azure siguiendo las mejores prácticas de seguridad, escalabilidad y rendimiento, con componentes claramente identificados para el proyecto vm-api-backend-appservice y vm-app-frontend.

1.1 Arquitectura de Despliegue

La solución está desplegada en la nube de Azure con la siguiente estructura:



1.2 Componentes de la Infraestructura del Proyecto VM

1.2.1 Frontend (vm-app-frontend)

- Azure Static Web App
- URL: <https://ambitious-field-02c5fb70f.6.azurestaticapps.net>
- Implementación mediante GitHub Actions para CI/CD
- Alojamiento de la aplicación Angular del Sistema de Gestión de VMs

- Interfaz de usuario responsive y moderna que se adapta a cualquier dispositivo

1.2.2 Backend (vm-api-backend-appservice)

- App Service:
- URL: <https://vm-api-backend-appservice-cvgmb8evdwh3cha5.canadacentral-01.azurewebsites.net>
- Aloja la API RESTful desarrollada en ASP.NET Core
- Expone endpoints para la gestión completa de máquinas virtuales
- Implementa comunicación en tiempo real mediante SignalR
- Integrado con seguridad JWT para autenticación y autorización

1.2.3 Base de Datos (mysql-horizonhub)

- Azure Database for MySQL
- Almacena toda la información del Sistema de Gestión de VMs
- Tablas optimizadas para el modelo de datos de máquinas virtuales
- Aislada dentro de la red virtual para mayor seguridad
- Accesible únicamente por el backend y la máquina virtual de administración

1.2.4 Máquina Virtual de Gestión (vm-horizon-hub)

- Azure VM
- Sirve como punto de acceso seguro para administración
- Permite realizar tareas de mantenimiento en la base de datos
- Dispone de herramientas de monitorización y diagnóstico
- Conectada a la red virtual con acceso privilegiado

1.2.5 Redes y Seguridad para el Sistema VM

Red Virtual (vnet-horizon-hub)

- Aísla todos los componentes del backend del Sistema de Gestión de VMs
- Segmentación por subredes para separar servicios

Grupo de Seguridad de Red (vm-horizon-hub-nsg)

- Reglas específicas para proteger los recursos del Sistema de VMs
 - Control de tráfico entrante y saliente con reglas granulares
 - Protección avanzada contra amenazas
-